

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_CMDS 16
#define MAX_ARGS 32
#define BUF_SIZE 1024

// Trim whitespace from string ends
char *trim(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Execute arbitrary-length multi-pipe command pipelines
void execute_pipeline(char *line) {
    char *cmds[MAX_CMDS];
    int num_cmds = 0;

    // Parse commands delimited by '|'
    char *token = strtok(line, "|");
    while (token != NULL && num_cmds < MAX_CMDS) {
        cmds[num_cmds++] = trim(token);
        token = strtok(NULL, "|");
    }

    if (num_cmds == 0) return;
```

```

// Allocate pipe file descriptors array: (N - 1) pipes require 2 * (N - 1) descriptors
int num_pipes = num_cmds - 1;
int pipefds[2 * num_pipes];

for (int i = 0; i < num_pipes; i++) {
    if (pipe(pipefds + i * 2) < 0) {
        perror("pipe allocation failed");
        return;
    }
}

pid_t pids[MAX_CMDS];

for (int i = 0; i < num_cmds; i++) {
    // Tokenize current command into arguments
    char *args[MAX_ARGS];
    int arg_count = 0;
    char *arg_token = strtok(cmds[i], " \\t");

    while (arg_token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = arg_token;
        arg_token = strtok(NULL, " \\t");
    }
    args[arg_count] = NULL;

    if (arg_count == 0) continue;

    pids[i] = fork();

    if (pids[i] < 0) {
        perror("fork failed");
        return;
    } else if (pids[i] == 0) {
        // Child Process: Redirect standard input if not first command
        if (i != 0) {
            if (dup2(pipefds[(i - 1) * 2], STDIN_FILENO) < 0) {
                perror("dup2 stdin failed");
            }
        }
    }
}

```

```

        exit(EXIT_FAILURE);
    }
}

// Child Process: Redirect standard output if not last command
if (i != num_cmds - 1) {
    if (dup2(pipefds[i * 2 + 1], STDOUT_FILENO) < 0) {
        perror("dup2 stdout failed");
        exit(EXIT_FAILURE);
    }
}

// Close all pipe descriptors inside child process
for (int j = 0; j < 2 * num_pipes; j++) {
    close(pipefds[j]);
}

execvp(args[0], args);
perror("execvp failed");
exit(EXIT_FAILURE);
}
}

// Parent Process: Close all pipe descriptors to send EOF properly
for (int i = 0; i < 2 * num_pipes; i++) {
    close(pipefds[i]);
}

// Synchronize and monitor termination of all launched children
for (int i = 0; i < num_cmds; i++) {
    int status;
    waitpid(pids[i], &status, 0);
}
}

int main() {
    char input_buf[BUF_SIZE];

```

```

    while (1) {
        printf("\nMultiPipe Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Remove newline

        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_pipeline(input_buf);
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano multi_pipe.c
abhis@MIKHAIL-37:~$ gcc -g -Wall multi_pipe.c -o multi_pipe
abhis@MIKHAIL-37:~$ ./multi_pipe

MultiPipe Shell > ls -la | grep multi
-rwxr-xr-x 1 abhis abhis 21272 Sep 29 10:08 multi_pipe
-rw-r--r-- 1 abhis abhis 3353 Sep 29 10:08 multi_pipe.c

MultiPipe Shell > cat /etc/passwd | cut -d: -f1 | head -n 5
root
daemon
bin
sys
sync

MultiPipe Shell > cat /etc/passwd | grep -v nologin | cut -d: -f1 | sort
abhis
dhcpcd
root
sync

MultiPipe Shell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./multi_pipe
==1750== Memcheck, a memory error detector
==1750== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1750== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1750== Command: ./multi_pipe
==1750==

MultiPipe Shell > exit
==1750==
==1750== HEAP SUMMARY:
==1750==      in use at exit: 0 bytes in 0 blocks
==1750==    total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==1750==
==1750== All heap blocks were freed -- no leaks are possible
==1750==
==1750== For lists of detected and suppressed errors, rerun with: -s
==1750== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
abhis@MIKHAIL-37:~$ nano multi_pipe.c
```